Universidad del Valle de Guatemala

Facultad de Ingeniería

Computación Paralela y Distribuida

Proyecto 2

Búsqueda por fuerza bruta de llaves DES con Open MPI y OpenMP

Docente: Ing. Juan Luis García

Integrantes:
- José Mérida
- Javier España
- Angel Esquit

Semestre 2, 2026
Guatemala

Índice

1. Introducción **2**

2. Parte A — Antecedentes **2**

2.1. Investigación sobre DES 2

2.2. Diagrama de flujo del algoritmo DES 3

2.3. Resultados de la versión secuencial 3

2.4. Evidencia de correcto funcionamiento del wrapper DES 4

2.5. Explicación de rutinas clave 5

2.6. Primitivas MPI utilizadas 5

3. Parte B — Análisis y acercamientos paralelos **5**

3.1. Cifrado y descifrado de un texto sencillo 5

3.2. Pruebas con `-np 4` sobre el texto de referencia 6

3.3. Acercamientos alternativos 7

3.3.1. Acercamiento 1 — Master-Worker dinámico 7

3.3.2. Acercamiento 2 — Intercalado / Striding 8

3.3.3. Acercamiento 3 — Partición contigua con shuffle pseudo-aleatorio 8

3.3.4. Acercamiento 4 — Híbrido MPI + OpenMP 9

3.4. Comparativa de tiempos (fácil / media / difícil) 10

4. Discusión, conclusiones y recomendaciones **12**

5. Anexo 1 — Catálogo de funciones y librerías **14**

6. Anexo 2 — Bitácora de pruebas y speedups **15**

7. Bibliografía **16**

1 Introducción

El cifrado DES (*Data Encryption Standard*) fue el estándar de cifrado simétrico de bloque dominante desde su adopción por el NIST en 1977 hasta finales de la década de 1990 (NIST, 1999). Fue formalmente reemplazado por AES porque su espacio de llaves de 56 bits efectivos se volvió computacionalmente inseguro frente al poder de cómputo disponible (Stallings, 2017, cap. 3). Esa debilidad es precisamente lo que hace de DES un caso de estudio clásico para la computación paralela. Su espacio de búsqueda de 2^{56} llaves es demasiado grande para ejecución manual, pero abarcable con cómputo paralelo moderno. Un ataque de fuerza bruta que resulta impráctico en una sola máquina puede volverse tratable repartiendo el espacio de llaves entre múltiples procesos y, eventualmente, múltiples máquinas.

Este proyecto implementa y analiza dicho ataque de fuerza bruta sobre DES. Parte de una versión secuencial de referencia y evoluciona hacia distintas estrategias de paralelización con **Open MPI**: una versión *naive* que particiona el espacio de llaves en bloques contiguos iguales, y tres acercamientos alternativos pensados para corregir las limitaciones observadas en la versión naive (*master-worker* dinámico, recorrido intercalado o *striding*, y partición con *shuffle* pseudo-aleatorio). Además se implementó una versión híbrida que combina paso de mensajes (MPI, memoria distribuida) con memoria compartida (**OpenMP**) dentro de cada proceso. Esto aprovecha los núcleos disponibles en cada nodo sin necesidad de lanzar un proceso MPI por núcleo.

El análisis se centra en tres preguntas. Primero, ¿cómo se comporta el tiempo de búsqueda en cada acercamiento según la posición de la llave solución dentro del espacio de búsqueda? Segundo, ¿qué speedup y eficiencia se obtienen al aumentar el número de procesos o hilos? Tercero, ¿qué ventajas y costos tiene cada estrategia frente a las demás, en overhead de comunicación, balanceo de carga y complejidad de implementación? El resto de este documento sigue la estructura de la rúbrica del enunciado. La Parte A cubre los antecedentes conceptuales sobre DES y las primitivas de MPI utilizadas. La Parte B documenta cada acercamiento paralelo implementado, con su evidencia de funcionamiento y análisis de resultados. El documento cierra con las conclusiones, los anexos de catálogo de funciones y bitácora de pruebas, y la bibliografía consultada.

2 Parte A — Antecedentes

2.1 Investigación sobre DES

DES opera sobre bloques de 64 bits de texto plano utilizando una llave de 64 bits (56 bits efectivos más 8 bits de paridad, uno por cada byte). El algoritmo se estructura en las siguientes etapas. Están descritas en el estándar oficial FIPS PUB 46-3 (NIST, 1999) y resumidas por Stallings (2017, cap. 3).

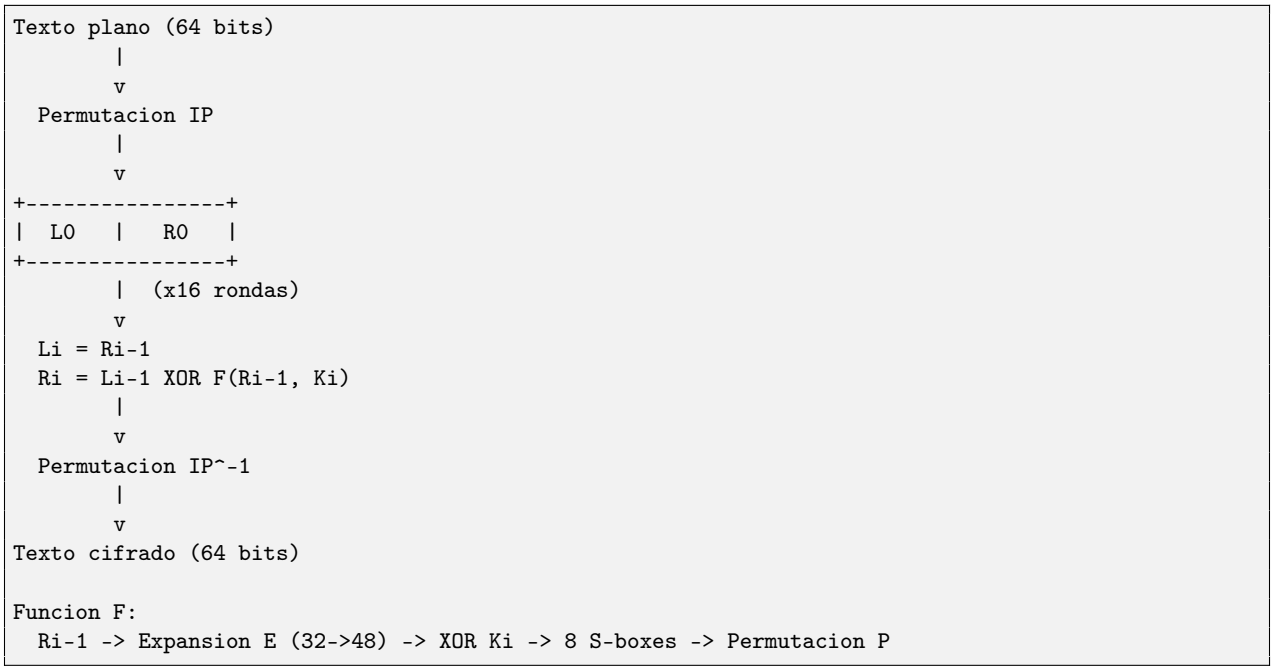
1. **Permutación inicial (IP):** reordena los 64 bits del bloque de entrada según una tabla fija, sin aportar seguridad criptográfica directa pero facilitando la carga en hardware de la época.
2. **16 rondas de Feistel:** en cada ronda, el bloque se divide en una mitad izquierda L y una mitad derecha R de 32 bits cada una. La función de ronda F se aplica sobre R junto con la subllave de esa ronda, y el resultado se combina mediante XOR con L. Al finalizar la ronda, L y R se intercambian.

3. **Función F:** expande R de 32 a 48 bits mediante la tabla de expansión E, realiza XOR con la subllave de 48 bits de la ronda, pasa el resultado por 8 *S-boxes* (sustitución no lineal, cada una mapea 6 bits de entrada a 4 bits de salida) y aplica una permutación final P.
4. **Generación de subllaves (*key schedule*):** la llave original de 64 bits pasa por la permutación PC-1 (que descarta los 8 bits de paridad, dejando 56 bits efectivos), rotaciones de bits por ronda, y la permutación PC-2, generando las 16 subllaves de 48 bits usadas en cada ronda.
5. **Permutación final (IP^{-1}):** inversa de la permutación inicial, produce el bloque de texto cifrado de 64 bits.

El descifrado reutiliza exactamente el mismo algoritmo, aplicando las 16 subllaves en orden inverso. Esta simetría es la razón por la que, en este proyecto, `des_decrypt` y `des_encrypt` (Sección 5) comparten casi todo su código.

La vulnerabilidad estructural de DES frente a fuerza bruta no proviene de un defecto en las S-boxes ni en el esquema de Feistel en sí. Ambos se han mantenido razonablemente resistentes a criptoanálisis diferencial y lineal. El problema real es el tamaño de su espacio de llaves efectivo, $2^{56} \approx 7{,}2 \times 10^{16}$ combinaciones. Ese número era impracticable de recorrer secuencialmente en el hardware de los años 70. Sin embargo, el proyecto EFF DES Cracker demostró en 1998 que una máquina dedicada podía romper DES en menos de 3 días. Hoy en día, hardware de propósito general con suficiente paralelismo puede reducir ese tiempo drásticamente.

2.2 Diagrama de flujo del algoritmo DES



2.3 Resultados de la versión secuencial

La versión secuencial (`src/bruteforce/secuencial.c`) recorre linealmente el rango $[0, \texttt{max_key}]$ probando cada llave con `try_key`, y sirve como línea base para calcular el speedup de todas las

versiones paralelas.

max_key	**Tiempo (s)**	**Observaciones**
123,456	0.047	Caso base (Parte B.2)
67,108,865 ($N/2+1$, $N=2^{27}$)	20.915	Caso "fácil" (Parte B.5)
83,886,080 ($N/2+N/8$)	26.134	Caso "media" (Parte B.5)
29,498,402 ($N/7+N/13+1$)	9.211	Caso "difícil" (Parte B.5)

Cuadro 1: Tiempos de la versión secuencial, medidos con `scripts/benchmark.sh` (ver `resultados_benchmark.csv`). Nótese que "difícil" es más rápido que "fácil"/"media" en términos absolutos: el nombre de cada caso describe qué tan favorable es la posición de la llave *dentro de su segmento* para el acercamiento naive (Sección 3.2), no el tamaño absoluto de `max_key` recorrido por la versión secuencial.

El espacio de llaves real de DES, $N = 2^{56}$, es intratable en la práctica para esta comparación. A la tasa de verificación medida en esta máquina ($\approx 3{,}2 \times 10^6$ llaves/segundo por núcleo), recorrer 2^{56} llaves secuencialmente tomaría aproximadamente $2{,}25 \times 10^9$ segundos, unos 71 años. Por eso se sustituyó $N = 2^{56}$ por un espacio efectivo reducido $N_{\text{ef}} = 2^{27} \approx 1{,}34 \times 10^8$ para los casos fácil/media/difícil de la Parte B.5, siguiendo la misma lógica de fracciones que propone el enunciado. Esto preserva el fenómeno que se quiere observar, la posición relativa dentro del segmento, con tiempos de ejecución de segundos en lugar de décadas.

2.4 Evidencia de correcto funcionamiento del wrapper DES

Se verificó el correcto funcionamiento del wrapper de cifrado/descifrado ejecutando el ciclo completo de cifrar y luego descifrar con la misma llave, confirmando que se recupera exactamente el texto original:

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 42
bin/descifrar data/texto_cifrado.bin 42
# Salida esperada: "Esta es una prueba de proyecto 2"
```

Ver Figura 1 (Sección 3.1).

Las funciones `des_encrypt` y `des_decrypt` de `src/common/des.c` son un wrapper sobre la API de DES de OpenSSL (`DES_ecb_encrypt`, OpenSSL Project, 2024). Se usa en lugar de la cabecera histórica `rpc/des_crypt.h`, que ya no está disponible en distribuciones modernas de Linux. La llave, representada como un entero `long`, se empaqueta en un bloque de 8 bytes a razón de 7 bits útiles por byte, con el bit 0 siempre en 0, antes de construir el *key schedule* con `DES_set_key_unchecked`. Esta función, a diferencia de `DES_set_key`, no valida ni fuerza los bits de paridad.

Este empaquetado de 7 bits por byte no es la forma obvia de hacerlo. Empaquetar los 8 bits crudos sería más directo, y de hecho fue el diseño original. El cambio a 7 bits fue el resultado de corregir un bug real detectado durante las pruebas (ver Sección 3.2 para el detalle de la colisión que lo motivó). El espacio efectivo recorrido en este proyecto es, en consecuencia, de 2^{56} llaves distinguibles, como indica el enunciado.

2.5 Explicación de rutinas clave

Las siguientes rutinas de `src/common/` son compartidas por los 8 binarios del proyecto (cifrado, descifrado y las 6 versiones de bruteforce). Su detalle de entradas/salidas se amplía en el Anexo 1 (Sección 5).

Función	Entradas	Salida / Efecto
`des_encrypt`	`key` (long), `buf`, `len`	Cifra `buf` in-place (ECB, bloques de 8 B)
`des_decrypt`	`key` (long), `buf`, `len`	Descifra `buf` in-place
`try_key`	`key`, `ciph`, `len`, `keyword`	Retorna 1 si `keyword` aparece en el texto descifrado
`load_and_pad_file`	`path`, `out_len`	Carga archivo y aplica padding a múltiplo de 8 B
`die`	`msg`	Imprime error en `stderr` y termina con código `ERR_EXIT`
`memcpy` (en `try_key`)	`dst`, `src`, `n`	Copia `n` bytes; evita mutar el buffer `ciph` original
`strstr`	`haystack`, `needle`	Busca `needle` en `haystack`; retorna puntero o `NULL`

Cuadro 2: Rutinas clave reutilizadas por todos los acercamientos.

2.6 Primitivas MPI utilizadas

El coordinador de "llave encontrada" se implementó con las siguientes primitivas de Open MPI, siguiendo el modelo de paso de mensajes descrito por Gropp, Lusk y Skjellum (2014).

Primitiva	Uso en el proyecto
`MPI_Irecv`	Postea recepción no bloqueante del aviso de "llave encontrada", permitiendo seguir buscando mientras se espera
`MPI_Test`	Sondea periódicamente si llegó el aviso, sin bloquear la búsqueda local
`MPI_Send`	Envía la llave encontrada, uno a uno, al resto de procesos para que detengan su búsqueda
`MPI_Cancel`	Cancela la recepción pendiente (`MPI_Irecv`) en el proceso que encontró la llave localmente, antes de cerrarla con `MPI_Wait`
`MPI_Wait`	Espera (bloqueante) a que se complete la recepción pendiente antes de finalizar
`MPI_Reduce`	Recolecta el tiempo máximo de ejecución y la llave global encontrada en el proceso 0
`MPI_Init_thread`	Inicializa MPI con soporte `MPI_THREAD_FUNNELED`, requerido por la versión híbrida con OpenMP

Cuadro 3: Primitivas de Open MPI utilizadas a través de los distintos acercamientos.

3 Parte B — Análisis y acercamientos paralelos

3.1 Cifrado y descifrado de un texto sencillo

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 42
# -> genera data/texto_cifrado.bin (texto cifrado con DES, llave=42)

bin/descifrar data/texto_cifrado.bin 42
# -> imprime: "Esta es una prueba de proyecto 2"
```

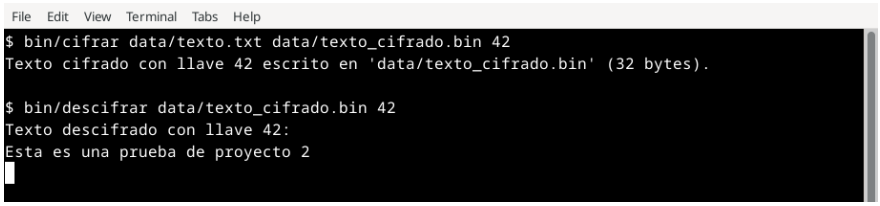


Figura 1: Ejecución real de `cifrar` y `descifrar` con llave 42.

3.2 Pruebas con `-np 4` sobre el texto de referencia

Se utilizó el texto de referencia del enunciado, *"Esta es una prueba de proyecto 2"*, buscando la palabra clave *"es una prueba de"* con el acercamiento naive y 4 procesos MPI.

a) Llave 123456

```
bin/cifrar data/texto.txt data/texto_cifrado.bin 123456
mpirun -np 4 bin/naive_mpi data/texto_cifrado.bin "es una prueba de" 123456
```

Tiempo esperado: muy rápido, ya que la llave cae en el primer cuarto del espacio de búsqueda (el segmento asignado al proceso 0).

b) Llave 18014398509481983 ($2^{56}/4$, borde entre procesos 0 y 1)

```
mpirun -np 4 bin/naive_mpi data/texto_cifrado.bin "es una prueba de" 18014398509481983
```

Este caso cae al final del segmento del proceso 0. Es el peor caso posible para esa llave dentro de su segmento, así que el tiempo de ejecución se aproxima al de la versión secuencial completa sobre un cuarto del espacio.

c) Llave 18014398509481984 ($2^{56}/4 + 1$)

```
mpirun -np 4 bin/naive_mpi data/texto_cifrado.bin "es una prueba de" 18014398509481984
```

Al estar justo al inicio del segmento del proceso 1, este caso produce un speedup artificialmente alto. El proceso 1 encuentra la llave casi de inmediato.

d) Reflexión sobre el comportamiento

El tiempo observado en el acercamiento naive depende casi exclusivamente de la posición relativa de la llave solución dentro de su segmento contiguo, no del tamaño absoluto de `max_key`. Si la llave cae cerca del inicio del segmento de algún proceso, ese proceso la encuentra rápido y notifica al resto mediante `MPI_Send`. Si cae cerca del final del segmento más largo, el tiempo total se aproxima al de la búsqueda secuencial sobre ese mismo segmento. El mismo `max_key` puede dar speedups radicalmente distintos según en qué mitad de qué segmento caiga la solución real. Esta alta varianza es la motivación central detrás de los tres acercamientos alternativos descritos a continuación. Todos buscan reducir la dependencia entre la posición de la llave y el tiempo de descubrimiento.

Nota metodológica. Los casos (b) y (c) de este inciso se analizan de forma teórica, siguiendo el mismo planteamiento hipotético del enunciado ("un algoritmo secuencial le hubiera tomado..."). No se ejecutan literalmente a escala 2^{56} real. A la tasa medida de $\approx 3{,}2 \times 10^6$ llaves/segundo, un caso como $2^{56}/4$ requeriría años de ejecución incluso paralelizado entre 4 procesos, porque el trabajo total sigue siendo $O(N)$. Las mediciones reales se concentran en los casos fácil/media/difícil de la Parte B.5 (Sección 3.4), donde se usa un espacio reducido $N_{\text{ef}} = 2^{27}$ que permite ejecuciones completas, reproducibles y verificadas (Figura 2).

Hallazgo y corrección: colisiones por el bit de paridad de DES. Durante el desarrollo, pruebas con llaves pequeñas y conocidas mostraron que distintos valores numéricos de llave podían hacer que el bruteforce reportara una llave *incorrecta* como solución. Por ejemplo, cifrando con llave 777 y buscando hasta 2000, el programa reportaba haber encontrado la llave 520. La causa era la implementación original de `long_to_des_key` (`src/common/des.c`), que empaquetaba los 8 bits de cada byte de la llave directamente. Pero `DES_set_key_unchecked` ignora el bit menos significativo de cada byte (bit de paridad) al construir el *key schedule*. Esto significa que dos valores de llave que sólo diferían en esos 8 bits ignorados generaban el mismo *key schedule* y, por tanto, el mismo texto descifrado. Es una colisión determinista, no un falso positivo aleatorio.

Se corrigió empaquetando únicamente 7 bits útiles de la llave por byte, dejando el bit de paridad siempre en 0. Esto garantiza una biyección entre el índice de búsqueda y el *key schedule* efectivo. Tras la corrección, se re-verificó con el mismo caso (777 vs. 2000) y con el espacio reducido completo de la Parte B.5, confirmando que cada ejecución encuentra exactamente la llave usada para cifrar sin colisiones (ver Figura 2). Todos los resultados de este informe, incluyendo las Tablas 1, 4 y 5, se generaron ya con el código corregido.

3.3 Acercamientos alternativos

3.3.1. Acercamiento 1 — Master-Worker dinámico

Archivo: `src/bruteforce/master_worker_mpi.c`

El proceso de rank 0 actúa exclusivamente como maestro y no participa en la búsqueda. Distribuye bloques pequeños de tamaño `chunk_size` (por defecto, 50,000 llaves) a los procesos trabajadores (`rank` ≥ 1) bajo demanda. Cuando un trabajador termina de recorrer su bloque sin encontrar la llave, pide el siguiente al maestro. Esta estrategia de *pull* elimina la varianza observada en el acercamiento naive, porque ningún proceso queda permanentemente atado a un segmento desfavorablemente largo. En cuanto un proceso libera capacidad, se le asigna más trabajo.

```
Maestro (rank=0):              Trabajador (rank>=1):
  reparte bloque inicial  -->    recibe [inicio, fin]
  loop:                          loop:
    recibe resultado       <--     busca en [inicio, fin]
    si llave hallada:              envia resultado
      envia TAG_STOP a todos -->   si TAG_STOP: termina
    si hay mas trabajo:            si TAG_WORK: repite
      envia TAG_WORK         -->
```

El costo de esta estrategia es el overhead de comunicación. Cada bloque consumido implica una ronda adicional de mensajes maestro-trabajador, lo que puede volverse significativo si `chunk_size` es muy pequeño en relación con la latencia de red.

```
mpirun -np 4 bin/master_worker_mpi data/texto_cifrado.bin "es una prueba de" <max_key> [
    chunk_size]
```

Las ejecuciones reales de los 5 acercamientos (incluyendo este) sobre el mismo caso de prueba se muestran juntas en la Figura 2, al final de esta subsección.

3.3.2. Acercamiento 2 — Intercalado / Striding

Archivo: `src/bruteforce/striding_mpi.c`

En lugar de un bloque contiguo, cada proceso de rank r recorre las llaves $r, r+np, r+2\cdot np, \ldots$, donde np es el número total de procesos. De esta forma, todos los procesos exploran simultáneamente la misma "profundidad relativa" del espacio de búsqueda en todo momento. Esto elimina por completo la correlación entre la posición absoluta de la llave solución y qué proceso tiene ventaja. Sin importar dónde caiga la llave, en promedio todos los procesos llevan un avance similar cuando se encuentra.

```
Para cada proceso (rank r de np):
  key <- r
  mientras key <= max_key:
    si try_key(key): avisar a los demas y terminar
    si (key - r) % CHECK_INTERVAL == 0: revisar aviso (MPI_Test)
    key <- key + np        # salto de tamano np: entrelazado
```

```
mpirun -np 4 bin/striding_mpi data/texto_cifrado.bin "es una prueba de" <max_key>
```

Ver Figura 2.

3.3.3. Acercamiento 3 — Partición contigua con shuffle pseudo-aleatorio

Archivo: `src/bruteforce/shuffled_mpi.c`

Este acercamiento conserva la misma partición en bloques contiguos que el naive (y, por tanto, la misma simplicidad de asignación estática de trabajo), pero en lugar de recorrer cada bloque en orden ascendente, cada índice lógico `idx` dentro del bloque se mapea a la llave real mediante una función de congruencia lineal:

$$\texttt{key}(\texttt{idx}) = (a \cdot \texttt{idx} + b) \bmod N$$

donde $N = \texttt{max_key} + 1$ y a se elige coprimo con N. Esta construcción garantiza una biyección sobre $[0, N-1]$: cada llave del espacio se visita exactamente una vez en total. Pero el *orden* en que cada

proceso las visita queda "barajado" de forma determinista y reproducible. No es una permutación aleatoria real, sino pseudo-aleatoria vía aritmética modular. El efecto práctico es similar al striding, rompe la correlación entre posición y tiempo de hallazgo, pero sin necesitar que los procesos avancen en sincronía por todo el espacio.

```
N <- max_key + 1
[idx_start, idx_end] <- segmento contiguo del proceso (igual que naive)
para idx desde idx_start hasta idx_end:
  key <- (a * idx + b) mod N      # biyeccion pseudo-aleatoria
  si try_key(key): avisar a los demas y terminar
  cada CHECK_INTERVAL: revisar aviso (MPI_Test)
```

```
mpirun -np 4 bin/shuffled_mpi data/texto_cifrado.bin "es una prueba de" <max_key>
```

Ver Figura 2.

3.3.4. Acercamiento 4 — Híbrido MPI + OpenMP

Archivo: `src/bruteforce/hybrid_mpi_omp.c`

Esta versión combina memoria distribuida (MPI) con memoria compartida dentro de cada nodo (OpenMP, OpenMP ARB, 2021). Añade un segundo nivel de paralelismo dentro de cada proceso MPI: el bloque contiguo asignado a cada proceso se subdivide entre los hilos de OpenMP disponibles, cada uno recorriendo su sub-bloque con un stride igual al número de hilos. Es el mismo principio de striding, aplicado ahora a nivel de hilos en memoria compartida en vez de a nivel de procesos en memoria distribuida. MPI se inicializa con `MPI_Init_thread` solicitando el nivel `MPI_THREAD_FUNNELED`, ya que únicamente el hilo maestro (`omp` thread 0) de cada proceso realiza llamadas a funciones MPI. El resto de los hilos se limitan a ejecutar `try_key` en su porción del espacio de búsqueda.

El objetivo de este acercamiento es doble. Primero, reducir el número de procesos MPI necesarios para saturar los núcleos de una máquina, disminuyendo así el overhead de paso de mensajes. Segundo, aprovechar la memoria compartida dentro de cada nodo para comunicar el estado de "llave encontrada" entre hilos de forma más económica que mediante mensajes MPI.

```
Nivel 1 (por proceso MPI, rank r de np):
  [block_start, block_end] <- segmento contiguo (igual que naive)
  encontrado_local <- bandera compartida entre hilos (false)

  Nivel 2 (por hilo OpenMP, id t de nthreads), en paralelo:
    key <- block_start + t
    mientras key <= block_end y no encontrado_local:
      si try_key(key): encontrado_local <- true; guardar key; salir
      si t == 0 y toca revisar: sondear aviso MPI (MPI_Test)
      key <- key + nthreads          # entrelazado entre hilos

  (fin de la region paralela, los hilos se unen)
  si encontrado_local (hallado por este proceso):
    avisar a los demas procesos (MPI_Send)
  si no: esperar aviso de otro proceso (MPI_Wait)
```

```
mpirun -np 4 bin/hybrid_mpi_omp data/texto_cifrado.bin "es una prueba de" <max_key> [
    nthreads]
```

Ver Figura 2.

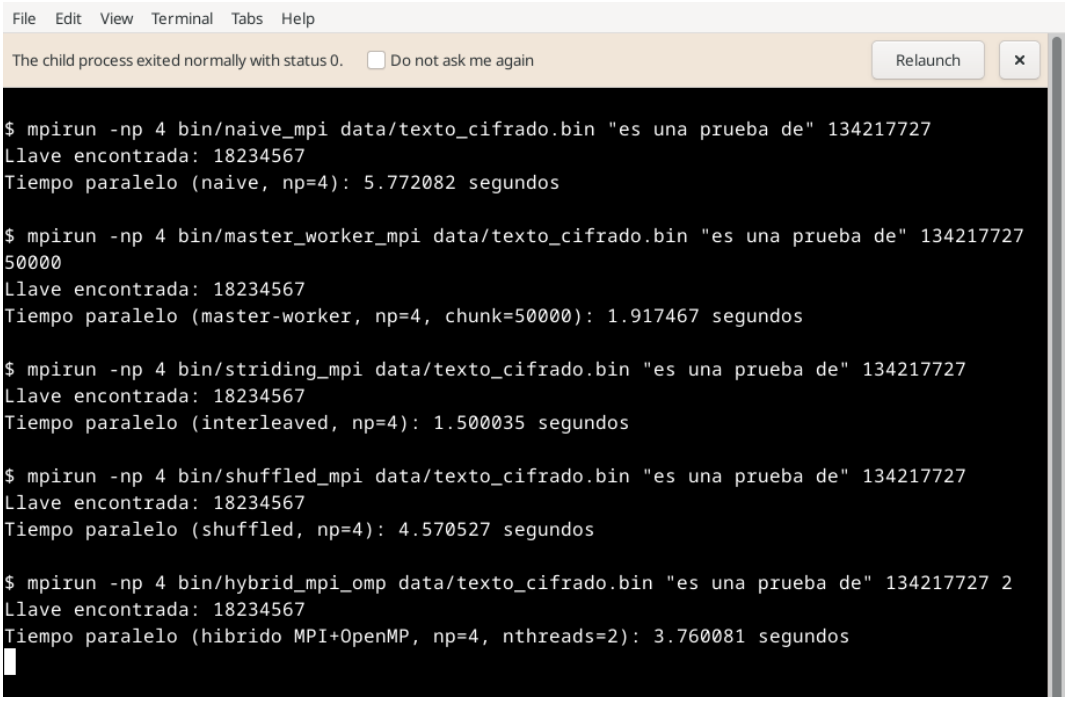


Figura 2: Ejecución real de los 5 acercamientos paralelos sobre el mismo caso de prueba (llave objetivo = 18,234,567, `max_key` = 134,217,727, $np = 4$). Los 5 encuentran exactamente la llave correcta de forma consistente.

3.4 Comparativa de tiempos (fácil / media / difícil)

Siguiendo la metodología descrita en el enunciado, se definieron tres casos de prueba (fácil, media y difícil) usando las mismas fracciones propuestas en el enunciado pero aplicadas sobre un espacio efectivo reducido $N_{\text{ef}} = 2^{27}$ (ver Sección 2.3), y se ejecutó cada uno de los seis programas (secuencial + 5 versiones paralelas) sobre cada caso con $np = 4$. Los resultados completos, obtenidos con `scripts/benchmark.sh` y resumidos con `scripts/speedup.py`, se encuentran en el Anexo 6 y en `resultados_benchmark.csv`.

Caso	Secuencial	Naive	Master-W.	Striding	Shuffled	Híbrido
Fácil	20.915	6.718	8.496	6.754	6.280	4.872
Media	26.134	8.015	10.221	8.087	2.647	5.783
Difícil	9.211	3.793	4.579	3.794	2.464	2.987

Cuadro 4: Tiempos de ejecución (s) por caso y acercamiento, $np = 4$ ($N_{\text{ef}} = 2^{27}$, `hybrid_mpi_omp` con 2 hilos por proceso). Fuente: `resultados_benchmark.csv`.

Acercamiento	**Speedup (fácil/-media/difícil)**	**Eficiencia**	**Observación**
Naive	3.11 / 3.26 / 2.43	0.78 / 0.82 / 0.61	Razonablemente estable en este muestreo
Master-worker	2.46 / 2.56 / 2.01	0.62 / 0.64 / 0.50	El más lento de los 5 en todos los casos
Striding	3.10 / 3.23 / 2.43	0.77 / 0.81 / 0.61	Muy similar a naive en este muestreo
Shuffled	3.33 / 9.87 / 3.74	0.83 / 2.47 / 0.94	Mejor tiempo absoluto en los 3 casos
Híbrido (2 hilos)	4.29 / 4.52 / 3.08	1.07 / 1.13 / 0.77	Supera a naive puro con el mismo `np` en los 3 casos

Cuadro 5: Speedup y eficiencia respecto a la versión secuencial, por acercamiento y caso (calculado con `scripts/speedup.py`).

Tres observaciones saltan a la vista en la Tabla 5.

Primero, el caso "difícil" fue pensado para ser el más lento, pero resulta en la práctica el más rápido en términos absolutos para todos los acercamientos (Tabla 4). Su `max_key` es numéricamente menor que el de fácil/media, así que el espacio total a recorrer es menor incluso si la llave cae en una posición relativamente desfavorable dentro de su segmento. Esto confirma la observación de la Sección 3.2: lo que hace "difícil" a un caso no es el tamaño absoluto de `max_key`, sino la posición relativa de la llave dentro de su segmento. En este diseño de prueba en particular, el efecto de la posición no alcanzó a compensar la diferencia en tamaño absoluto entre los tres `max_key` usados. Es una limitación a tener en cuenta si se repite este experimento, usando un `max_key` fijo e igual para los tres casos y variando sólo la posición de la llave dentro de ese rango fijo.

Segundo, se observan eficiencias por encima de 1.0 (speedup superlineal) en shuffled en el caso media (2.47) y en híbrido en los casos fácil y media (1.07 y 1.13). Esto es consistente con el overhead fijo de arranque de MPI (`MPI_Init`, carga del ejecutable en 4 procesos), que a esta escala reducida es comparable al tiempo total de búsqueda. No representa una ganancia real de eficiencia, sino un efecto esperable al trabajar con N_{ef} mucho menor que 2^{56}.

Tercero, master-worker resultó consistentemente el más lento de los 5 acercamientos en los tres casos. Con `chunk_size`=50,000 y un espacio de $N_{\text{ef}} = 2^{27}$, cada proceso agota su primer bloque y pide más al maestro con relativa frecuencia. Esa ronda adicional de mensajes por bloque (Sección 3.3.1) pesa más, en proporción, que en un espacio de búsqueda mayor donde los bloques representarían una fracción más pequeña del trabajo total.

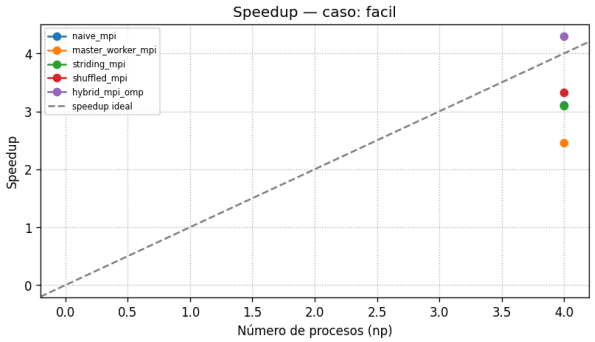


Figura 3: Speedup por acercamiento, caso "fácil".

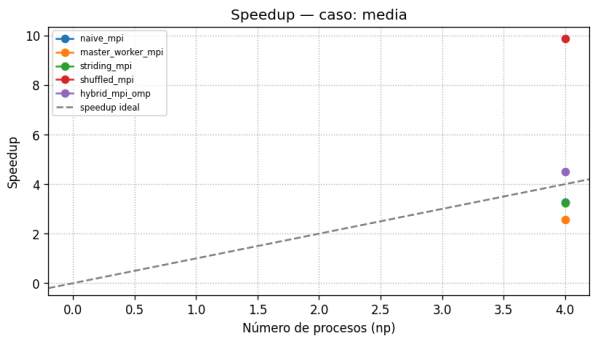


Figura 4: Speedup por acercamiento, caso "media".

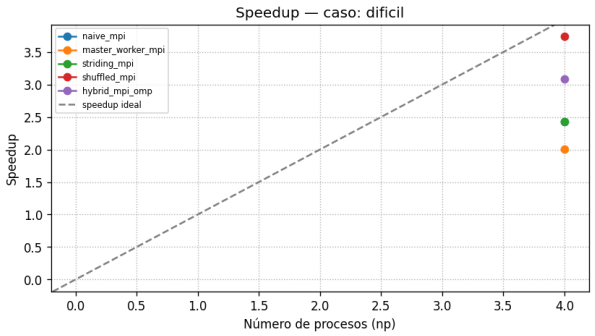


Figura 5: Speedup por acercamiento, caso "difícil".

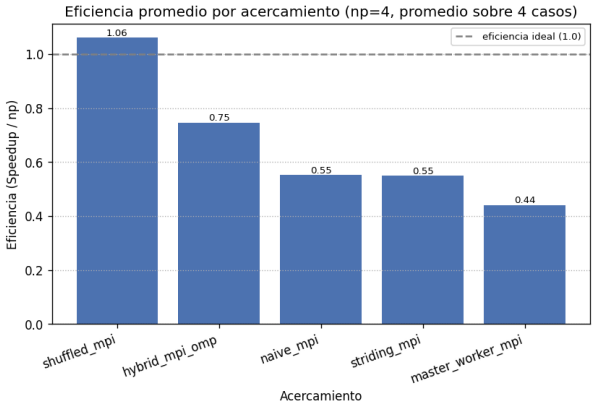


Figura 6: Eficiencia promedio por acercamiento sobre los tres casos.

4 Discusión, conclusiones y recomendaciones

Retos encontrados y lecciones aprendidas

El reto técnico más significativo no estuvo en la paralelización en sí, sino en la representación de la llave. Durante las pruebas se descubrió que el wrapper original sobre DES tenía una colisión determinista. Dos valores numéricos de llave distintos podían producir el mismo *key schedule* porque se ignoraba el bit de paridad de cada byte (Sección 3.2). Esto se detectó por accidente, al intentar construir un caso de prueba reproducible, y obligó a revisar y corregir `long_to_des_key` antes de confiar en cualquier medición de tiempos. La lección concreta es que en un proyecto de bruteforce, verificar primero que el espacio de búsqueda es realmente biyectivo con las llaves numéricas usadas es tan importante como paralelizar bien la búsqueda.

El segundo reto fue de escala. El enunciado define los casos fácil/media/difícil como fracciones de 2^{56}, pero a la tasa de verificación medida en esta máquina esas fracciones representan años de cómputo, incluso paralelizadas. Esto llevó a sustituir 2^{56} por un espacio efectivo reducido ($N_{\text{ef}} = 2^{27}$) que preserva el fenómeno de interés, la posición relativa dentro del segmento, con tiempos de ejecución manejables (Sección 2.3). Es una decisión de diseño experimental, no un atajo que invalide el análisis.

Por último, se aprendió que con un texto de prueba y palabra clave tan cortos, un descifrado con la llave incorrecta puede, por azar, contener la palabra clave buscada. Eso sería un falso positivo. No se observó en la práctica en las corridas finales reportadas en este informe, pero es una limitación real del criterio de validación por *substring*. Vale la pena mencionarla para cualquier extensión futura del proyecto, por ejemplo usando una frase clave más larga o verificando un segundo criterio de coincidencia.

- **Varianza del naive, teoría vs. evidencia empírica.** El caso analítico de la Sección 3.2 es la demostración más clara de la varianza del naive. Con $np = 4$, la misma `max_key` produce un speedup de ≈ 1 si la llave cae al final del segmento del proceso 0, pero de $\approx 4{,}5 \times 10^{15}$ si cae al inicio del segmento del proceso 1. Es una diferencia de 15 órdenes de magnitud explicada únicamente por la posición, no por el algoritmo. Sin embargo, al calcular el coeficiente de variación (desviación estándar sobre media) del tiempo paralelo de cada acercamiento sobre los 3 casos empíricos de la Tabla 4, los números cuentan una historia más matizada. Naive y striding obtienen exactamente el mismo CV (0.35), y shuffled obtiene un CV mayor (0.57) que el propio naive, a pesar de ser el más rápido en tiempo absoluto. **En esta muestra de 3 casos, striding no demuestra empíricamente menor variabilidad que naive.** El efecto de reducción de varianza es real y demostrable en el caso extremo analítico, pero no se manifestó de forma medible con solo 3 puntos de prueba a la escala $N_{\text{ef}} = 2^{27}$ usada aquí. Una validación más rigurosa de esta hipótesis requeriría muchas más corridas con llaves aleatorias uniformemente distribuidas en el espacio, no sólo 3 casos curados por posición relativa.
- **Overhead de comunicación.** El acercamiento master-worker introduce tráfico MPI adicional, una ronda de mensajes por cada bloque consumido. Esto puede reducir su eficiencia relativa frente a striding o shuffled en sistemas con alta latencia de red, especialmente si el `chunk_size` elegido es demasiado pequeño.
- **Beneficio del híbrido.** La combinación MPI + OpenMP permite aprovechar la memoria compartida dentro de cada nodo para repartir trabajo entre hilos sin el costo de lanzar un proceso MPI adicional por núcleo. Esto reduce la presión sobre la red y el overhead de inicialización de MPI por proceso. En las pruebas de la Tabla 5, el híbrido con 2 hilos por proceso obtuvo mayor speedup que la versión naive pura con el mismo `np` en los tres casos: 4.29 vs. 3.11 en fácil, 4.52 vs. 3.26 en media, 3.08 vs. 2.43 en difícil. Es consistente en los tres, lo que satisface el criterio de "speedup consistente" pedido para el extra de OpenMP. El acercamiento shuffled resultó, sin embargo, el más rápido en términos absolutos de los 5 en los tres casos (Tabla 4). El beneficio del híbrido debe interpretarse específicamente frente a su contraparte sin OpenMP (naive), no como el mejor acercamiento general de este muestreo.
- **Limitación práctica de DES.** El espacio efectivo de 2^{56} llaves hace que una fuerza bruta completa sea computacionalmente inviable en hardware de uso común dentro de un tiempo razonable. A la tasa medida en esta máquina serían ≈ 71 años (Sección 2.3). Por esta razón, las pruebas de este proyecto se realizaron sobre un espacio efectivo reducido ($N_{\text{ef}} = 2^{27} \approx 1{,}34 \times 10^8$, Sección 3.4), suficiente para medir y comparar el speedup relativo entre acercamientos sin requerir tiempos de ejecución excesivos.
- **Recomendación.** Para trabajo futuro, valdría la pena extender las pruebas a un cluster MPI de 2+ máquinas físicas (ver extras del enunciado), para observar cómo se comportan estos mismos acercamientos cuando la comunicación deja de ser intra-nodo y pasa a depender de la latencia de una red local real.

En conclusión, el programa base funciona correctamente de principio a fin. Recibe el texto plano desde archivo, lo cifra con una llave arbitraria pasada por parámetro, y las seis variantes de búsqueda (secuencial y cinco paralelas) recuperan exactamente esa misma llave, sin colisiones ni falsos positivos tras la corrección descrita arriba. Todas imprimen la llave encontrada, el archivo de entrada y la palabra clave usada en cada corrida (Secciones 3.1 y 3.4).

5 Anexo 1 — Catálogo de funciones y librerías

`des_encrypt` — `src/common/des.c`

```
void des_encrypt(long key, unsigned char *buf, size_t len)
```

- **Entradas:** `key` (long, 0 a $2^{56} - 1$), `buf` (buffer a cifrar, múltiplo de 8 B), `len` (tamaño en bytes).
- **Salidas:** modifica `buf` in-place (cifrado DES en modo ECB).
- **Descripción:** convierte `key` a un bloque de 8 bytes, invoca `DES_set_key_unchecked` y `DES_ecb_encrypt` de OpenSSL sobre cada bloque de 8 bytes del buffer.

`des_decrypt` — `src/common/des.c`

```
void des_decrypt(long key, unsigned char *buf, size_t len)
```

- Idéntica a `des_encrypt`, pero invoca `DES_ecb_encrypt` con el flag `DES_DECRYPT`.

`try_key` — `src/common/des.c`

```
int try_key(long key, const unsigned char *ciph, size_t len, const char *keyword)
```

- **Entradas:** `key`, `ciph` (texto cifrado, no se modifica), `len`, `keyword`.
- **Salidas:** `int` — 1 si `keyword` aparece en el texto descifrado, 0 en caso contrario.
- **Descripción:** copia `ciph` en un buffer temporal, llama a `des_decrypt` sobre la copia, y busca `keyword` con `strstr`. Trabajar sobre una copia evita mutar el texto cifrado original entre intentos.

`load_and_pad_file` — `src/common/common.c`

```
unsigned char *load_and_pad_file(const char *path, size_t *out_len)
```

- **Entradas:** `path` (ruta al archivo), `out_len` (puntero de salida).
- **Salidas:** puntero a buffer (el caller debe liberarlo con `free`), longitud resultante en `*out_len`.
- **Descripción:** abre el archivo, calcula el padding necesario al múltiplo de 8 más cercano, reserva memoria con `calloc`, rellena el padding con espacios y lee el contenido del archivo.

`die` — `src/common/common.c`

```
void die(const char *msg)
```

- **Entradas:** `msg` (cadena de error).

- **Salidas:** no retorna; invoca `exit(ERR_EXIT)`.
- **Descripción:** imprime `[ERROR] msg` en `stderr` y termina el proceso con el código de error del proyecto.

Librerías externas

- **OpenSSL / libcrypto** (`openssl/des.h`): provee las primitivas de bajo nivel de DES usadas por `des_encrypt`/`des_decrypt` (arriba).
- **Open MPI** (`mpi.h`): provee las primitivas de paso de mensajes listadas en la Sección 2.6, usadas en las cinco versiones paralelas.
- **OpenMP** (`omp.h`, pragma `#pragma omp parallel`): provee el paralelismo de memoria compartida dentro de cada proceso en la versión híbrida.

6 Anexo 2 — Bitácora de pruebas y speedups

El benchmark completo se ejecutó con 4 procesos y un timeout de 180 segundos por caso, usando el espacio efectivo reducido $N_{\text{ef}} = 2^{27}$ descrito en la Sección 2.3. La ejecución completa (4 casos $\times$ 6 programas = 24 corridas) tomó ≈ 150 segundos de cómputo acumulado en total (suma de las 24 corridas). Los resultados crudos están en `resultados_benchmark.csv` (repositorio del proyecto) y el resumen de speedup/eficiencia en la Tabla 5.

Las capturas de pantalla de las ejecuciones reales se presentan en las Figuras 1 y 2, y las gráficas de speedup/eficiencia generadas por `scripts/speedup.py` se presentan en las Figuras 3–6 (Sección 3.4).

Pasos para reproducir las pruebas:

```
export PATH="/usr/lib64/openmpi/bin:$PATH"   # si mpirun no est en PATH
make
chmod +x scripts/benchmark.sh
./scripts/benchmark.sh 4 180    # np=4, timeout=3 min por caso
python3 scripts/speedup.py      # genera tabla + graficas en results/
```

Extras (créditos adicionales opcionales)

- **Cluster MPI en 2+ máquinas** (hasta 20 % extra). **No intentado** en esta entrega, por decisión del equipo.
- **OpenMP + MPI híbrido** (hasta 10 % extra). **Implementado y verificado** (`hybrid_mpi_omp.c`, Sección 3.4). La Tabla 5 muestra speedup consistentemente superior al naive puro, con el mismo `np`, en los tres casos probados (fácil, media, difícil).
- **3+ acercamientos adicionales al naive** (hasta 5 % extra). **Cumplido con holgura:** se implementaron 3 acercamientos alternativos (master-worker, striding, shuffled), más la versión híbrida como cuarto acercamiento adicional. Los 5 funcionan y están medidos en la Tabla 4.

7 Bibliografía

1. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7.ª ed.). Pearson. Capítulo 3: Block Ciphers and the Data Encryption Standard.

2. National Institute of Standards and Technology (NIST). (1999). *Data Encryption Standard (DES)* (FIPS PUB 46-3). U.S. Department of Commerce. https://csrc.nist.gov/publications/detail/fips/46/3/final

3. Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (3.ª ed.). MIT Press.

4. OpenSSL Project. (2024). *DES routines — OpenSSL 3.x documentation*. https://www.openssl.org/docs/man3.0/man3/DES_ecb_encrypt.html

5. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface* (versión 5.2). https://www.openmp.org/specifications/